

棋盘游戏管理系统

这是一个支持多种棋盘游戏模式的管理系统，目前支持的游戏模式包括：

- 1. **和平模式** - 简单地放置棋子，不遵循特定规则
- 2. **黑白棋模式** - 经典的黑白棋（翻转棋）规则
- 3. **五子棋模式** - 15x15棋盘的五子棋，包含障碍物和炸弹道具

项目结构

该项目采用面向对象设计模式，主要包含以下类：

— ReversiGame.java	# 程序入口点
— Game.java	# 游戏流程控制
— Board.java	# 棋盘状态管理
— BoardManager.java	# 多棋盘管理
— Player.java	# 玩家信息管理
— GameMode.java	# 游戏模式接口
— PeaceMode.java	# 和平模式实现
— ReversiMode.java	# 黑白棋模式实现
— GomokuMode.java	# 五子棋模式实现
— BoardDisplay.java	# 棋盘显示接口
— ClassicBoardDisplay.java	# 8x8棋盘显示
— GomokuBoardDisplay.java	# 15x15棋盘显示
— BoardDisplayFactory.java	# 棋盘显示工厂

设计模式

该项目使用了多种设计模式：

- 1. **策略模式** - 不同的游戏模式和显示实现
- 2. **工厂模式** - 通过BoardDisplayFactory创建适当的显示器
- 3. **状态模式** - 在BoardManager中管理多个棋盘状态

特色功能

玩家体验优化

- 固定玩家名称为"张三"和"李四"，简化游戏启动流程
- 切换棋盘后无需按任意键确认，可直接输入落子位置
- 支持使用"exit"或"quit"命令退出游戏
- 优化的界面布局，确保不同游戏模式下的信息对齐显示

五子棋模式特性

- 15x15棋盘
- 行号采用16进制表示(1-F)，列号采用字母表示(A-O)
- 特定位置的固定障碍物(3F、8G、9F、CK)
- 炸弹道具：黑方2个，白方3个
- 使用炸弹格式：@位置，例如 @FA （炸弹可以清除对方棋子）
- 自定义错误提示：障碍物提示"该位置有障碍物！"，已占用提示"该位置已经有棋子占据！"
- 胜利条件：五子连珠

和平模式特性

- 初始化时与黑白棋模式相同，带有中间四个棋子
- 界面中隐藏得分显示，保持简洁的用户界面

多棋盘管理

- 可以创建多个不同模式的棋盘
- 可以自由切换棋盘进行游戏
- 每个棋盘独立保存游戏状态

棋谱回放功能

- 支持读取棋谱文件自动播放棋局
- 使用简洁的命令格式：playback 文件名
- 每步间隔1秒自动下棋，显示"自动下棋中"状态
- 按任意键可以停止回放
- 游戏结束或棋谱播放完毕后自动返回到主界面

如何运行

1. 编译所有Java文件：

```
javac *.java
```

2. 运行主程序：

```
java ReversiGame
```

3. 游戏会自动以"张三"和"李四"作为玩家名称开始

游戏操作指南

基本操作

- 输入坐标下棋，例如： 1a 代表第1行a列
- 输入数字切换棋盘，例如： 1 切换到1号棋盘
- 输入游戏模式名称创建新棋盘，如 peace 、 reversi 、 gomoku
- 输入 exit 或 quit 退出游戏
- 输入 playback 文件名 播放棋谱文件

五子棋模式特殊操作

- 输入16进制行号和字母列号，例如： 1A 代表第1行A列
- 输入 @位置 使用炸弹，例如： @FA 在F行A列使用炸弹

核心类设计思路

1. ReversiGame 类

作为程序入口，这个类变得非常简洁，只负责启动游戏：

```
public class ReversiGame {  
    public static void clearScreen() {  
        // 清屏逻辑  
    }  
  
    public static void main(String[] args) {  
        // 使用固定的玩家名称  
        String player1Name = "张三";  
        String player2Name = "李四";  
  
        Game game = new Game(player1Name, player2Name);  
        game.start();  
    }  
}
```

2. GameMode 接口

通过接口定义不同游戏模式的行为规范，是策略模式的应用：

```
public interface GameMode {  
    boolean isValidMove(Board board, int row, int col, Board.Piece piece);  
    boolean placePiece(Board board, int row, int col, Board.Piece piece);  
    boolean hasValidMoves(Board board, Board.Piece piece);  
}
```

每种游戏模式都必须实现这三个基本方法，保证了游戏逻辑的一致性和可扩展性。

3. Board 类

负责棋盘状态的维护和显示，是游戏的核心数据结构：

```

public class Board {
    public enum Piece { EMPTY, BLACK, WHITE, BARRIER, CRATER }
    private Piece[][] board;
    private int size = 15; // 默认五子棋模式为15x15棋盘

    // 初始化不同模式的棋盘方法
    public void initializeReversiBoard() {...}
    public void initializeEmptyBoard() {...}
    public void initializePeaceBoard() {...} // 初始化和平模式棋盘（带初始棋子）
    public void initializeGomokuBoard() {...}

    // 特殊功能方法
    public void setBarrier(int row, int col) {...}
    public void setCrater(int row, int col) {...}

    // 棋盘操作和状态查询方法
    public Piece getPiece(int row, int col) {...}
    public void setPiece(int row, int col, Piece piece) {...}
    public boolean isFull() {...}
    public int[] getScore() {...}

    // 棋盘显示
    public void printBoard(...) {...}
}

```

4. 游戏模式实现类

4.1 ReversiMode（传统黑白棋）

实现了传统黑白棋的规则，包括判断移动有效性、放置棋子并翻转对手棋子等：

```

public class ReversiMode implements GameMode {
    @Override
    public boolean isValidMove(Board board, int row, int col, Board.Piece piece) {
        // 实现黑白棋有效移动的判断逻辑
    }

    @Override
    public boolean placePiece(Board board, int row, int col, Board.Piece piece) {
        // 实现放置棋子并翻转对手棋子的逻辑
    }

    @Override
    public boolean hasValidMoves(Board board, Board.Piece piece) {
        // 检查是否还有有效移动
    }
}

```

4.2 PeaceMode（和平模式）

简化版的棋类规则，只需要棋子放在空位即可：

```

public class PeaceMode implements GameMode {
    @Override
    public boolean isValidMove(Board board, int row, int col, Board.Piece piece) {
        return row >= 0 && row < 8 && col >= 0 && col < 8 &&
            board.getPiece(row, col) == Board.Piece.EMPTY;
    }

    // 其他方法实现...
}

```

4.3 GomokuMode（五子棋模式）

实现了五子棋的规则，特别是连子判断和炸弹功能：

```

public class GomokuMode implements GameMode {
    // 错误类型常量
    public static final int ERROR_NONE = 0;
    public static final int ERROR_BARRIER = 1;
    public static final int ERROR_OCCUPIED = 2;

    private int moveCount = 0;
    private int blackBombs = 2; // 黑方有2个炸弹
    private int whiteBombs = 3; // 白方有3个炸弹
    private int lastError = ERROR_NONE;

    // 基本方法实现...

    // 获取当前轮数
    public int getRoundCount() {
        return (moveCount / 2) + 1;
    }

    // 获取最后的错误类型
    public int getLastError() {
        return lastError;
    }

    // 使用炸弹
    public boolean useBomb(Board board, int row, int col, Board.Piece currentPiece) {
        // 检查炸弹数量
        // 清除对方棋子
        // 设置弹坑
    }

    // 核心方法：检查是否有五子连珠
    public boolean checkWin(Board board, int row, int col, Board.Piece piece) {
        // 实现四个方向（水平、垂直、两个斜向）的连子判断
    }
}

```

5. BoardManager 类

管理多个棋盘，支持棋盘切换和多棋盘状态维护：

```
public class BoardManager {  
    private ArrayList<Board> boards = new ArrayList<>();  
    private ArrayList<GameMode> modes = new ArrayList<>();  
    private ArrayList<Player> currentPlayers = new ArrayList<>();  
    private int currentBoardIndex = -1;  
  
    // 添加新棋盘  
    public int addBoard(GameMode mode) {...}  
  
    // 切换棋盘  
    public boolean switchBoard(int index) {...}  
  
    // 获取当前棋盘信息  
    public Board getCurrentBoard() {...}  
    public GameMode getCurrentMode() {...}  
    public Player getCurrentPlayer() {...}  
  
    // 管理玩家  
    public void setCurrentPlayer(Player player) {...}  
    public void setPlayerAtIndex(int index, Player player) {...}  
}
```

6. Game 类

负责整体游戏流程控制，是游戏的核心控制类：


```

public class Game {
    private final BoardManager boardManager = new BoardManager();
    private final Player player1;
    private final Player player2;
    private ArrayList<Boolean> boardFinished = new ArrayList<>();

    public Game(String p1, String p2) {
        // 初始化玩家、棋盘等
    }

    public void start() {
        // 游戏主循环
        while (true) {
            // 1. 显示当前棋盘
            // 2. 检查胜负条件
            // 3. 处理玩家输入
            // 4. 更新游戏状态
        }
    }

    /**
     * 从文件中读取棋谱并自动播放
     * @param filename 棋谱文件名
     */
    private void playbackFromFile(String filename) {
        // 棋谱回放逻辑
    }
}

```

7. BoardDisplay 与显示相关类

通过策略模式实现了不同棋盘的显示方式：

```

public interface BoardDisplay {
    void displayBoard(Board board, Player currentPlayer, Player player1, Player player2,
        GameMode mode, int boardIndex, BoardManager boardManager);
}

```

其中，ClassicBoardDisplay负责8x8棋盘的显示，针对Peace模式隐藏了分数显示；GomokuBoardDisplay则负责15x15棋盘的特殊显示。

实现亮点

1. **玩家体验优化**：通过游戏流程优化，减少不必要的交互，如切换棋盘后可直接操作，固定玩家名称，提供多种退出命令等。
2. **自适应界面**：针对不同游戏模式优化了界面显示，确保Peace模式下的游戏列表正确对齐。
3. **棋谱回放功能**：支持从文件读取棋谱并自动播放，支持任意时刻停止回放，回放结束后自动返回游戏界面。
4. **策略模式的应用**：通过 GameMode 接口定义游戏规则，不同模式作为具体策略实现，使游戏模式可以灵活切换和扩展。
5. **责任分离**：将不同功能分配到各个类中，每个类有明确的职责，提高了代码的可维护性。
6. **多棋盘管理**：通过 BoardManager 实现了多棋盘的管理和切换，玩家可以同时进行多局游戏。
7. **错误提示细化**：在五子棋模式中，提供了针对不同情况的错误提示，增强了用户体验。

扩展性设计

项目设计考虑了良好的扩展性：

1. 要添加新的游戏模式，只需创建新类实现 GameMode 接口
2. 通过 BoardManager 可以轻松管理更多的棋盘
3. 所有游戏逻辑通过接口解耦，修改一种模式不会影响其他模式

功能改进记录

1. 玩家名称固定化

将玩家名称固定为"张三"和"李四"，简化了游戏启动流程。

```
// ReversiGame.java
public static void main(String[] args) {
    // 使用固定的玩家名称
    String player1Name = "张三";
    String player2Name = "李四";

    Game game = new Game(player1Name, player2Name);
    game.start();
}
```

2. 五子棋错误信息优化

针对五子棋模式添加了自定义错误类型，提供更精确的错误提示：

```
// GomokuMode.java
public static final int ERROR_NONE = 0;
public static final int ERROR_BARRIER = 1;
public static final int ERROR_OCCUPIED = 2;

// Game.java中的相关处理
if (errorType == GomokuMode.ERROR_BARRIER) {
    System.out.println("该位置有障碍物！");
} else if (errorType == GomokuMode.ERROR_OCCUPIED) {
    System.out.println("该位置已经有棋子占据！");
} else {
    System.out.println("无效的移动！");
}
```

3. 退出命令增强

增加了"quit"作为"exit"的替代命令：

```
if (input.equalsIgnoreCase("exit") || input.equalsIgnoreCase("quit")) {
    return;
}
```

4. 棋谱回放功能

实现了从文件读取棋谱并自动播放的功能：

```

// 接收命令
else if (input.startsWith("PLAYBACK ")) {
    // 提取文件名
    String filename = input.substring(9).trim();
    playbackFromFile(filename);
    continue;
}

// 回放方法
private void playbackFromFile(String filename) {
    // 读取文件，逐步执行棋步，支持在任意时刻按键停止
}

```

5. Peace模式初始化

修改Peace模式初始化，与黑白棋相同，带有中间四个棋子：

```

// Board.java
public void initializePeaceBoard() {
    board = new Piece[8][8];
    for (int i = 0; i < 8; i++)
        for (int j = 0; j < 8; j++)
            board[i][j] = Piece.EMPTY;

    // 添加与黑白棋模式相同的初始棋子
    board[3][3] = Piece.WHITE;
    board[3][4] = Piece.BLACK;
    board[4][3] = Piece.BLACK;
    board[4][4] = Piece.WHITE;
}

```

6. 界面优化

针对Peace模式隐藏了分数显示，保持了游戏列表的对齐：

```
// ClassicBoardDisplay.java
if (mode instanceof PeaceMode) {
    System.out.printf("玩家[%s] %s          1号棋盘: peace",
        player1.getName(), mark);
} else {
    System.out.printf("玩家[%s] %s 得分: %-4d          1号棋盘: peace",
        player1.getName(), mark, score[0]);
}
```

7. 交互流程优化

切换棋盘或创建新棋盘后，不再需要按任意键继续，可以直接输入下一步操作：

```
// 删除了以下代码
// System.out.println("按任意键继续...");
// scanner.nextLine();
```

总结

通过面向对象设计和模块化重构，将原本单一文件的程序拆分为多个具有明确职责的类，使代码结构更加清晰，便于维护和扩展。策略模式的应用使得游戏模式可以灵活切换，提高了系统的灵活性和可扩展性。

本次改进重点关注用户体验优化，包括简化交互流程、提供友好的错误提示、优化界面显示以及添加实用功能如棋谱回放。这些改进使游戏操作更加流畅直观，提高了玩家体验。